

3D 人脸识别模组 通信协议说明

版本：V1.5

产品文档修订记录

版本	说明
V0.1	创建文档, 添加基本内容
V1.0	添加串口配置, 消息的发送和接受
V1.1	更新通信协议说明, 修订格式
V1.2	添加对体验模式的说明
V1.3	附件 2 中添加加密详细说明
V1.4	重复注册 ITG 命令支持人脸方向
V1.5	增加特征值注册指令

目录

1 串口配置	5
2 通信消息格式	5
3 通信消息详细说明	6
3.1 消息头文件定义	6
3.2 消息列表	6
4 模块消息的发送和接收	8
4.1 消息接收(H>>M)	8
4.2 消息发送(M>>H)	12
5 主控接收消息流程	18
6 一般消息处理流程	19
7 上下电流程	20
8 录入流程	21
9 录入说明	22
10 单帧录入	23
11 解锁流程	23
12 体验模式	24
13 加密通信流程（可不使用，由客户决定）	25
14 OTA 升级流程	26
15 补充说明	27
15.1 MID_REPLY	27
15.2 MID_NOTE	28
15.3 MID_RESET	28
15.4 MID_GETSTATUS	28
15.5 MID_VERIFY	28
15.6 MID_ENROLL/MID_ENROLL_SINGLE	29
15.7 MID_ENROLL_ITG	30
15.8 MID_ENROLL_BY_FEATURE	31
15.9 MID_DELUSER	31
15.10 MID_DELALL	31
15.11 MID_GETUSERINFO	31

15.12 MID_FACERESET	32
15.13 MID_POWERDOWN	32
15.14 MID_DEMOMODE	32
15.15 MID_GET_ALL_USERID	32
15.16 MID_SET_RELEASE_ENC_KEY	32
15.17 MID_INIT_ENCRYPTION	32
15.18 MID_SET_THRESHOLD_LEVEL	33
附件 1	34
附件 2	41

1 串口配置

模组和主控之间采用串口方式通信，配置如下：

表 1

配置项	说明
波特率	支持大多波特率，默认 115200
硬件/软件流控制	不使用
数据位	8
停止位	1
奇偶校验位	不使用

2 通信消息格式

主控与模组通信的基本消息格式如表 2 所示

表 2

SyncWord	MsgID	Size	Data	ParityCheck
2 bytes	1 byte	2 bytes	N bytes	1 byte

表 3 是对各个字段的详细说明。

表 3

SyncWord	2 bytes	固定的消息开头同步字 0xEF 0xAA
MsgID	1 byte	消息 ID (例如 RESET)
Size	2 bytes	Data size, 单位 byte
Data	N bytes	消息对应的 data, 如 command 消息对应的参数。 65535>=N>=0, N=0 表示此消息无参数。
ParityCheck	1 byte	协议的奇偶校验码，计算方式为整条协议除去 SyncWord 部分后，其余字节按位做 XOR 运算。

3 通信消息详细说明

3.1 消息头文件定义

消息头文件定义如附件 1 所示。

3.2 消息列表

表 4 中列出了模块(M)和主控(H)的所有通信消息，每个消息包含 ID 和消息 data 的定义(详见附件头文件 message.h)。每条消息处理都对应有 timeout 超时时间定义，timeout 指的是主控等待模块处理的最长时间，即模块反馈处理完成的时间。**灰色部分暂未使用。**

当指令处理超时，主控可采用如下几种处理方式：

1. 发送 MID_GETSTATUS 指令此消息能快速获取模块的工作状态，若死机则无返回，主控可直接断电。
2. 发送 RESET 指令停止所有当前的处理，模块进入 standby 的状态，等待主控新的指令。
3. 认为模块死机主控可采用断电重启的处理方式。

表 4

MsgID	Code	方向	Data 结构	TimeOut (ms)	说明
MID_REPLY	0x00	M»H	s_msg_reply_data	N/A	Reply 是模块对主控发送出的命令的应答，对于主控的每条命令，模块最终都会进行 reply
MID_NOTE	0x01	M»H	s_msg_note_data	N/A	Note 是模块主动发给主控的信息，根据 note id 判断消息类型和适配的 data 结构
MID_RESET	0x10	H»M	N/A	1000	停止所有当前在处理的命令，模块进入 standby 状态
MID_GETSTATUS	0x11	H»M	N/A	200	立即返回模块当前状态
MID_VERIFY	0x12	H»M	s_msg_verify_data	10000	鉴权解锁
MID_ENROLL	0x13	H»M	s_msg_enroll_data	10000	交互录入
MID_ENROLL_SINGLE	0x1D	H»M	s_msg_enroll_data	10000	单帧录入
MID_ENROLL_BY_FEATURE	0x1F	H»M	s_msg_enroll_feature_data	10000	特征值录入
MID_DELUSER	0x20	H»M	s_msg_deluser_data	200	删除一个注册用户
MID_DELALL	0x21	H»M	N/A	200	删除所有注册用户
MID_FACERESET	0x23	H»M	N/A	200	重置算法状态，如果正在进行交互式录入，会清空已录入的方向
MID_GET_ALL_USERID	0x24	H»M	N/A	500	获取所有已注册用户的 d 数量和 ID
MID_ENROLL_ITG	0x26	H»M	s_msg_enroll_itg	10000	集成支持并扩展所有录入方式
MID_GET_VERSION	0x30	H»M	N/A	1000	获得软件版本信息

MID_START_OTA	0x40	H»M	s_msg_startota_data	N/A	进入 OTA 升级模式
MID_STOP_OTA	0x41	H»M	N/A	N/A	退出 OTA 模式，模组重启。
MID_GET_OTA_STATUS	0x42	H»M	N/A	N/A	获取 OTA 状态以及传输升级包的起始包序号
MID_OTA_HEADER	0x43	H»M	s_msg_otaheader_data	N/A	发送升级包的大小，总包数，分包的大小，升级包的 md5 值
MID_OTA_PACKET	0x44	H»M	s_msg_otapacket_data	N/A	发送升级包：包序号，包大小，包数据。
MID_INIT_ENCRYPTION	0x50	H»M	s_msg_init_encryption_data	1000	设置加密随机数
MID_CONFIG_BAUDRATE	0x51	H»M	s_note_config_baudrate	100	OTA 模式下设定通信口波特率
MID_SET_RELEASE_ENC_KEY	0x52	H»M	s_msg_enc_key_number_data	N/A	设定量产加密秘钥序列，掉电会保存
MID_SET_DEBUG_ENC_KEY	0x53	H»M	s_msg_enc_key_number_data	N/A	设定调试加密秘钥序列，掉电丢失
MID_SET_THRESHOLD_LEVEL	0xD4	H»M	s_msg_algo_threshold_level	N/A	设置算法安全等级
MID_POWERDOWN	0xED	H»M	N/A	1000	模块准备关机，结束当前在处理的任務，清除文件系统缓存
MID_DEMOMODE	0xFE	H»M	N/A	100	进入演示模式

4 模块消息的发送和接收

4.1 消息接收(H>>M)

模块接收到的完整协议格式如表 5 所示，主控应按照该格式向模块发送命令。

表 5

名称	SyncWord	MsgID	Size		Data	ParityCheck
字节数	2 bytes	1 byte	2 bytes		N bytes	1 byte
内容	0xEFAA	command	高八位	低八位	data	checksum

command 和 data 定义如表 6 所示。

表 6

Command (*表示携带 data)	Code	Data		说明
		结构体	内容	
RESET	0x10	无		无
GET STATUS	0x11	无		无
VERIFY*	0x12	s_msg_verify_data	pd_rightaway (1byte)	解锁成功后是否立刻断电 (yes:1 no:0)
			Timeout (1byte)	解锁超时时间 (单位 s)
ENROLL	0x13	s_msg_enroll_data	admin	是否设置为管理员 (yes:1 no:0)
			user_name (32bytes)	录入用户姓名
			s_face_dir (1byte)	用户需要录入的方向，具体参数如表 7 所示
			timeout (1 byte)	录入超时时间 (单位 s)
ENROLL_SINGLE	0x1D	s_msg_enroll_data	admin	是否设置为管理员 (yes:1 no:0)
			user_name (32bytes)	录入用户姓名
			s_face_dir (1byte)	不使用
			timeout (1 byte)	录入超时时间 (单位 s)

ENROLL_BY_FEATURE	0x1F	s_msg_enroll_feature_data	admin	是否设置为管理员 (yes:1 no:0)
			user_name (32bytes)	录入用户姓名
			s_face_dir (1byte)	不使用
			timeout (1byte)	录入超时时间（单位 s）
			feature (512byte)	录入特征点
DELETE USER*	0x20	s_msg_deluser_data	user_id_heb (1byte)	待删除用户的 ID
			user_id_leb (1byte)	
DELETE ALL	0x21	无		删除所有用户
FACE RESET	0x23	无		清除录入状态
MID_GET_ALL_USERID	0x24	无		获取所有已注册用户的数量和 ID
MID_ENROLL_ITG	0x26	s_msg_enroll_itg	admin	是否设置为管理员 (yes:1 no:0)
			user_name (32bytes)	录入用户姓名
			face_dir (1byte)	人脸方向
			enroll_type (1 byte)	注册类型：交互录入或单帧录入
			enable_duplicate (1 byte)	是否重复录入
			Timeout (1 byte)	录入超时时间（单位 s）
			Reserved (3 byte)	预留位
GET VERSION	0x30	无		获取软件版本
MID_START_OTA*	0x40	s_msg_startota_data	v_primary (1 byte)	固件版本号
			v_secondary (1 byte)	
			v_revision (1 byte)	
MID_OTA_HEADER*	0x43	s_msg_otaheader_data	fsz_b (4 bytes)	固件大小，高位在前 B0 b1 b2 b3

			num_pkt (4 bytes)	总包数，高位在前 B0 b1 b2 b3
			pkt_size (2 bytes)	每包大小，高位在前 B0 b1
			md5_sum (32 bytes)	固件 MD5 值
MID_OTA_PACKET*	0x44	s_msg_otapacket_data	Pid (2 bytes)	分包序号
			Psize (2 bytes)	分包大小
			Data (n byts)	分包内容
INITENCRYPTIO*	0x50	s_msg_init_encryption_data	seed[4] (4 bytes)	主控发送一个随机数
			Mode (1 bytes)	加密模式
MID_CONFIG_BAUDRATE	0x51	s_msg_config_baudrate	baudrate_index	波特率 1:115200 2:230400 3:460800 4:1500000
MID_SET_RELEASE_ENC_KEY	0x52	s_msg_enc_key_number_data	enc_key_number[16]	加密序列
MID_SET_DEBUG_ENC_KEY	0x53	s_msg_enc_key_number_data	enc_key_number[16]	加密序列
POWER DOWN	0xED	无		无
DEMO MODE*	0xFE	s_msg_demomode_data	Enable (1byte)	enable:1 disable: 0

人脸方向定义如表 7 所示。

表 7

st_face_dir (人脸方向定义)	code	说明
FACE_DIRECTION_UP	0x10	录入朝上的人脸
FACE_DIRECTION_DOWN	0x08	录入朝下的人脸
FACE_DIRECTION_LEFT	0x04	录入朝左的人脸
FACE_DIRECTION_RIGHT	0x02	录入朝右的人脸
FACE_DIRECTION_MIDDLE	0x01	录入正向的人脸
FACE_DIRECTION_UNDEFINE	0x00	未定义，默认为正向

人脸录入命令类型定义如表 8 所示。

表 8

itg enroll_type (人脸录入命令类型)	code	说明
FACE_ENROLL_TYPE_INTERACTIVE	0x00	交互录入
FACE_ENROLL_TYPE_SINGLE	0x01	单帧录入

示例： 0xEF 0xAA 0x10 0x00 0x00 0x10

名 称	SyncWord	MsgID	Size		Data	ParityCheck
内 容	0xEF00AA	0x10 (MID_RESET)	0x00	0x00	无	0x10

这条消息是主控发送给模块的 RESET 消息，数据为长度为 0。

4.2 消息发送(M>>H)

模块主要发送两种类型的消息，分别是 REPLY， NOTE。

(1) REPLY 消息的发送

(2) 模块向主控发送的 REPLY 消息的完整协议如表 8 所示。

表 8

名称	SyncWord	MsgID	Size		Data			ParityCheck
字节数	2 bytes	1 byte	2 bytes		N bytes			1 byte
内容	0xEFAA	MID_REPLY (0x00)	高八位	低八位	s_msg_reply_data			checksum
					Mid (1byte)	Result (1byte)	data[0] (n-byte)	

mid 表示模块当前正在处理的任务，例如当 mid = MID_ENROLL(0x13)时，表示该消息是模块处理完 enroll 任务后回复的消息。 mid 详细如表 9 所示。

表 9

Mid (*表示携带 data)	Code	Data		说明
		结构体	内容	
MID_RESET	0x10	无		无
MID_GETSTATUS*	0x11	s_msg_repy_getstatus_data	Status (1byte)	模块的状态包括： IDLE(0),BUSY(1) ERROR(2),INVALID(3)
MID_VERIFY* (只有当 result 结果为 MR_SUCCESS 时才有 user_id)	0x12	s_msg_reply_verify_data	user_id_heb (1byte)	已验证用户的 ID
			user_id_leb (1byte)	
			user_name (32 bytes)	用户名字
			Admin (1byte)	是否为管理员
			unlockStatus (1 byte)	暂未使用
MID_ENROLL (只有当 result 结果为 MR_SUCCESS 时才有 user_id)	0x13	s_msg_reply_enroll_data	user_id_heb (1byte)	已注册用户的 ID
			user_id_leb (1byte)	
			face_direction (1byte)	各个方向人脸的录入状态

MID_ENROLL_SINGLE (只有当 result 结果为 MR_SUCCESS 时才有 user_id)	0x1d	s_msg_reply_enroll_data	user_id_heb (1byte)	已注册用户的 ID
			user_id_leb (1byte)	
			face_direction (1byte)	各个方向人脸的录入状态
MID_ENROLL_BY_FEATURE (只有当 result 结果为 MR_SUCCESS 时才有 user_id)	0x1f	s_msg_reply_enroll_data	user_id_heb (1byte)	已注册用户的 ID
			user_id_leb (1byte)	
			face_direction (1byte)	各个方向人脸的录入状态
MID_DELUSER	0x20	无		无
MID_DELALL	0x21	无		无
MID_FACERESET	0x23	无		无
MID_GET_ALL_USERID*	0x24	s_msg_reply_all_userid_data	user_counts (1 byte)	已注册用户数量
			users_id[40] (40 bytes)	所有已注册用户 ID, 使用连续两个字节存储一个 ID, 先存高八位
MID_ENROLL_ITG*	0x26	s_msg_reply_enroll_data	user_id_heb (1byte)	已注册用户的 ID
			user_id_leb (1byte)	
MID_GET_VERSION*	0x30	s_msg_reply_version_data	version_info[32] (32 bytes)	版本信息
MID_START_OTA	0x40	无		无
MID_STOP_OTA	0x41	无		无
MID_GET_OTA_STATUS	0x42	s_msg_reply_getotastatus_data	ota_status (1 byte)	4 表示处于 OTA 状态中
			next_pid_e (2 bytes)	传输固件包的起始序号
MID_OTA_HEADER	0x43	无		无
MID_OTA_PACKET	0x44	无		无

MID_INIT_ENCRYPTION	0x50	s_msg_reply_init_encryption_data	device_id[20] (20 bytes)	设备 id 信息
MID_CONFIG_BAUDRATE	0x51	无		无
MID_SET_RELEASE_ENC_KEY	0x52	无		无
MID_SET_DEBUG_ENC_KEY	0x53	无		无
MID_SET_THRESHOLD_LEVEL	0xD4	无		无
MID_POWERDOWN	0xED	无		无
MID_DEMOMODE	0xFE	无		无

result 表示该命令的执行结果，详细如表 10 所示。

Result	code	说明
MR_SUCCESS	0	成功
MR_REJECTED	1	模块拒绝该命令
MR_ABORTED	2	录入/解锁算法已终止
MR_FAILED4_CAMERA	4	相机打开失败
MR_FAILED4_UNKNOWNREASON	5	未知错误
MR_FAILED4_INVALIDPARAM	6	无效的参数
MR_FAILED4_NOMEMORY	7	内存不足
MR_FAILED4_UNKNOWNUSER	8	没有已录入的用户
MR_FAILED4_MAXUSER	9	录入超过最大用户数量
MR_FAILED4_FACEENROLLED	10	人脸已录入
MR_FAILED4_LIVENESSCHECK	12	活体检测失败
MR_FAILED4_TIMEOUT	13	录入或解锁超时
MR_FAILED4_AUTHORIZATION	14	加密芯片授权失败
MR_FAILED4_READ_FILE	19	读文件失败
MR_FAILED4_WRITE_FILE	20	写文件失败
MR_FAILED4_NO_ENCRYPT	21	通信协议未加密

示例： 0xEF 0xAA 0x00 0x00 0x04 0x13 0x00 0x00 0x01 0x11

名称	SyncWord	MsgID	Size		Data				Parity Check
内容	0xEFAA	0x00 (MID_REPLY)	0x00	0x04	s_msg_reply_data				0x11
					0x13 (MID_ENROLL)	0x00 (MR_SUCCESS)	0x00 (user_id_hex)	0x01 (user_id_leb)	

这条消息表示这是模块回复给主控的 REPLY 消息，数据部分占 4 字节，录入成功且录入的用户 ID 是 1。

(3) NOTE 消息的发送

NOTE 消息主要作用是主动向主控返回一些信息，目前 NOTE 消息主要在三种情况下发送：

- a)开机时模块向主控发送 NID_READY；
- b)录入过程中模块向主控发送人脸信息；
- c)解锁过程中模块向主控发送人脸信息。

模块向主控发送的 NOTE 消息的完整协议如表 11 所示。

表 11

名称	SyncWord	MsgID	Size		Data		ParityCheck
字节数	2 bytes	1 byte	2 bytes		N bytes		1 byte
内容	0xEFAA	MID_NOTE (0x01)	高 八 位	低 八 位	s_msg_note_data		checksum
					Nid (1byte)	data[0] (n-bytes)	

nid 表示 enroll 过程中算法的执行结果，详细如下：

表 12

nid (*表示携带 data)	code	说明
NID_READY	0	模块已准备好
NID_FACE_STATE*	1	算法执行成功，并且返回人脸信息 s_note_data_face
NID_UNKNOWNERROR	2	未知错误
NID_OTA_DONE*	3	OTA 升级完毕

NID_FACE_STATE 所携带的数据 data 主要存储了人脸信息，详细如下：

结构	s_note_data_face							
内容	State	Left	Top	Right	Bottom	Yaw	Pitch	Roll
类型	int16_t	int16_t	int16_t	int16_t	int16_t	int16_t	int16_t	int16_t
字节	2 bytes	2 bytes	2 bytes	2 bytes	2 bytes	2 bytes	2 bytes	2 bytes

state:表示人脸当前的状态，主要包括以下几种情况：

人脸状态(state)	code	说明
-------------	------	----

FACE_STATE_NORMAL	0	人脸正常
FACE_STATE_NOFACE	1	未检测到人脸
FACE_STATE_TOOUP	2	人脸太靠近图片上边沿，未能录入
FACE_STATE_TOODOWN	3	人脸太靠近图片下边沿，未能录入
FACE_STATE_TOOLEFT	4	人脸太靠近图片左边沿，未能录入
FACE_STATE_TOORIGHT	5	人脸太靠近图片右边沿，未能录入
FACE_STATE_FAR	6	人脸距离太远，未能录入
FACE_STATE_CLOSE	7	人脸距离太近，未能录入
FACE_STATE_EYEBROW_OCCLUSION	8	眉毛遮挡
FACE_STATE_EYE_OCCLUSION	9	眼睛遮挡
FACE_STATE_FACE_OCCLUSION	10	脸部遮挡
FACE_STATE_DIRECTION_ERROR	11	录入人脸方向错误

s_note_data_face 中其它成员的具体含义如下所示：

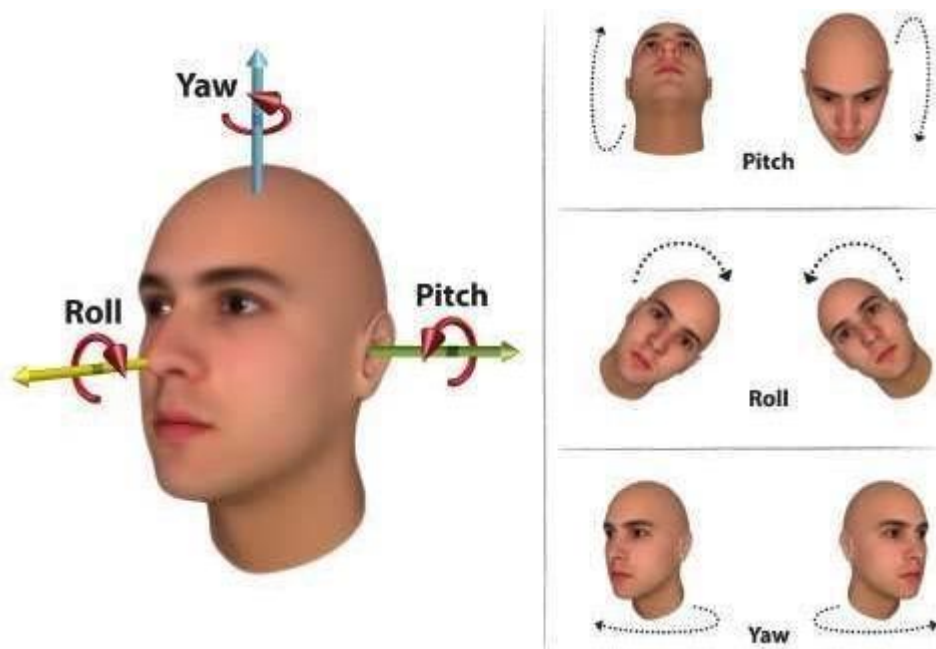
left:人脸框距离图片最左侧的距离（负数表示人脸框已超出图片最左侧）

top:人脸框距离图片最上方的距离（负数表示人脸框已超出图片上方）

right:人脸框距离图片最右侧的距离（负数表示人脸框已超图片最右侧）

bottom:人脸框距离图片最下 方的距离（负数表示人脸框已超出图片最下方）

yaw,pitch,roll 表示的旋转方向分别如下图所示。其中 yaw 为负表示左转头，yaw 为正表示右转头； pitch 为负表示向上抬头， pitch 为正表示向下低头；roll 为负表示向右歪头， roll 为正表示向左歪头。



示例： 0xEF 0xAA 0x01 0x00 0x01 0x00 0x00

名称	SyncWord	MsgID	Size		Data		ParityCheck
字节数	2 bytes	1 byte	2 bytes		N bytes		1 byte
内容	0xEFAA	0x01 (MID_NOTE)	0x00	0x01	s_msg_note_data 0x00 (NID_READY)	无	0x00

这条消息表示这是模块向主控发送的 NOTE 消息，数据长度为 1 字节，模块已经准备好接收其他命令。

5 主控接收消息流程

主控接收模块发送的消息可以遵循图 1 所示流程。

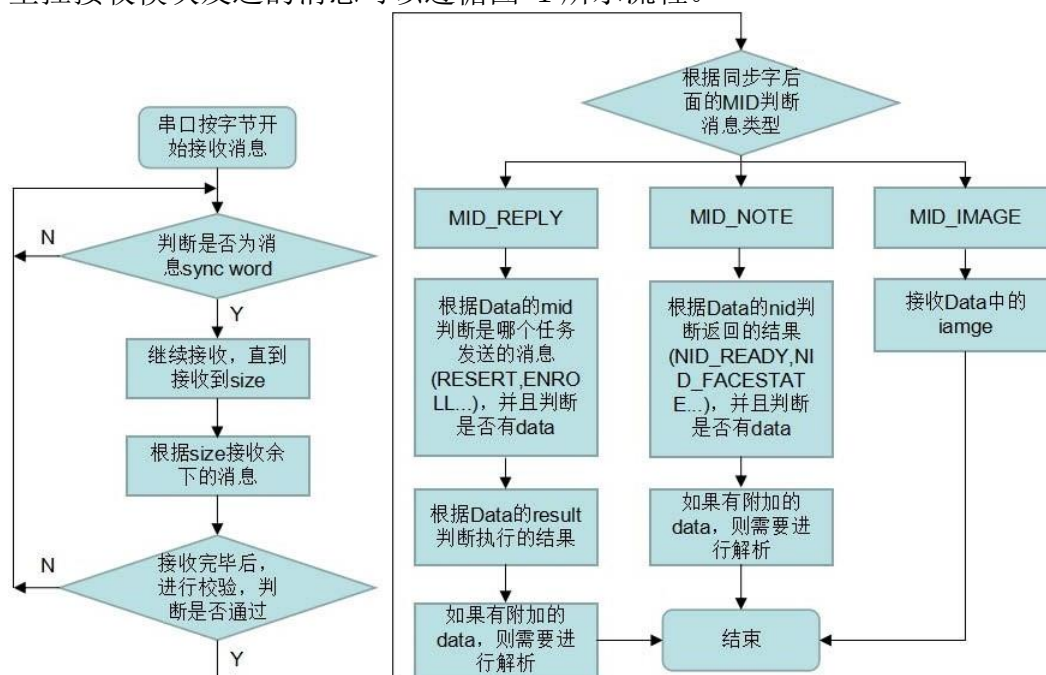


图 1

6 一般消息处理流程

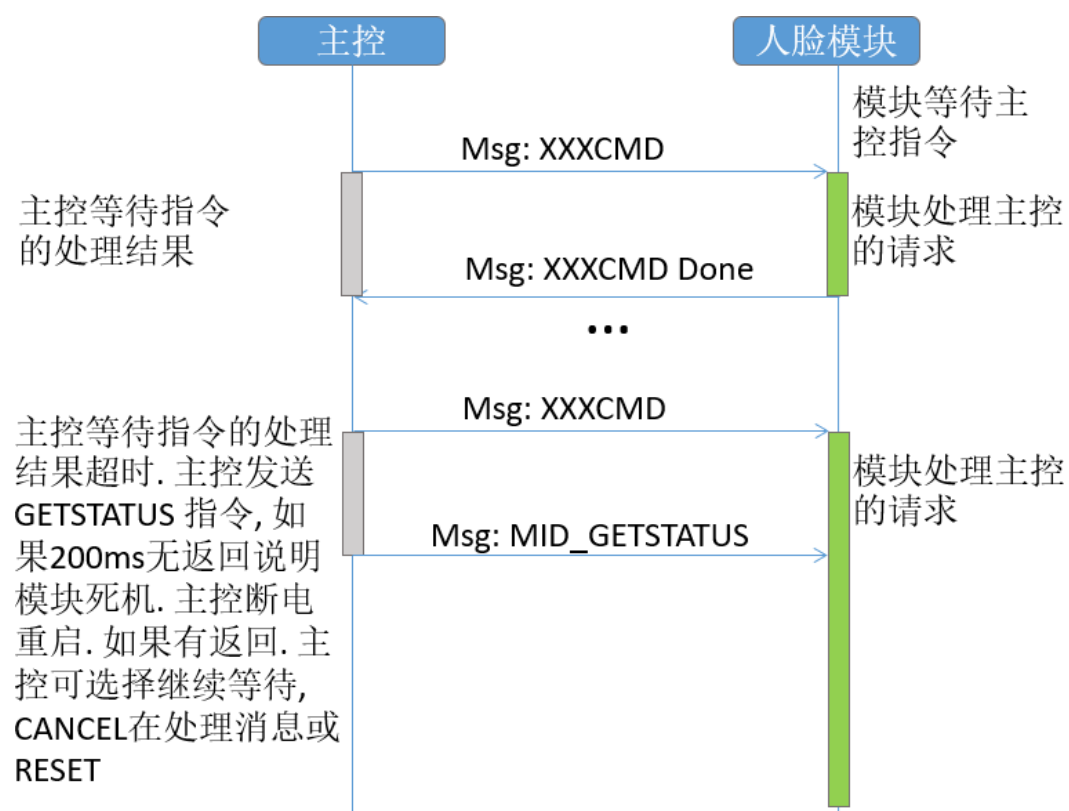


图 2

7 上下电流程

主控主动控制模块的上电和下电，如图 4-3 所示。模块上电即进入启动流程，启动进入 standby 状态会通知主控 MSG::NOTE::READY，主控开始发送命令消息，模块处理并返回结果。无消息发送时，主控调用 MSG::POWERDOWN，模块做处理返回，主控可断电。



图 3

8 录入流程

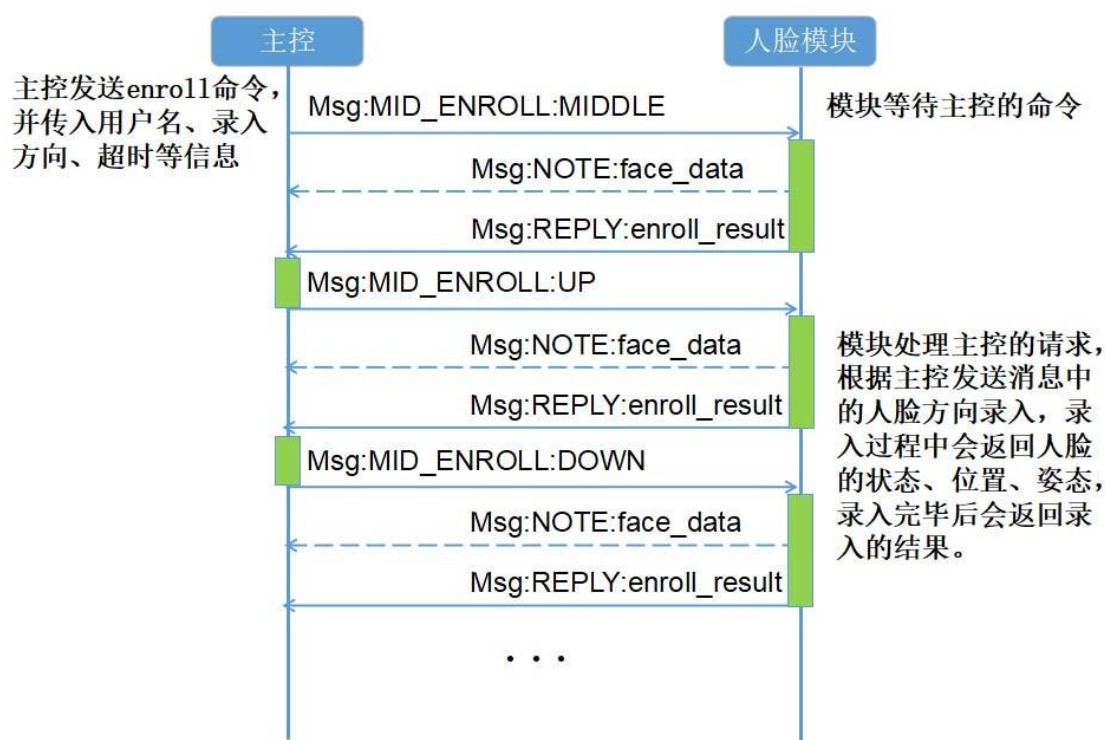


图 4

在录入过程中，人脸模块不限制录入方向的顺序，用户可以任意组合录入方向的顺序，图 4-4 是以其中一种顺序为例进行说明。

当主控下达录入某个方向的人脸的指令后，模块开始工作，并实时地返回人脸的状态、位置、姿态等信息（以 NOTE 消息的形式返回），用户可根据该信息提醒录入人进行姿态位置的调整，当录入成功后，模块将以 REPLY 消息的形式返回录入结果，如果录入不成功，模块也会相应地返回失败的原因。

值得注意的是，当模块拿到的第一帧图片就录入成功时，将直接以 REPLY 消息的形式返回结果。

录入过程中可通过 FACE RESET 指令终止录入，先前的录入状态也会清零。

如果在录入过程中突然断电，之前录入的人脸也不会保存。

9 录入说明

人脸录入各个方向的角度说明如下：

录入方向	偏转角度
正脸	正对摄像头，无偏转角度
向上	正脸向上偏转 5~55 度
向下	正脸向下偏转 5~55 度
向右	正脸向右偏转 8~60 度
向左	正脸向左偏转 8~60 度

其中，向上和向下偏转如图 5 所示。



图 5

向左和向右偏转如图 6 所示。



图 6

请不要采用如图 4-7 所示的转头方式。



图 7

我们建议用户在实际录入的过程中，缓缓向某个方向转头，并且保持偏转，请不要过快的转头或者立刻回归正脸状态。

10 单帧录入

单帧录入相比录入流程的区别，主要在于单帧录入只需要一张正脸即可录入成功。而不需要交互五次完成注册。

11 解锁流程

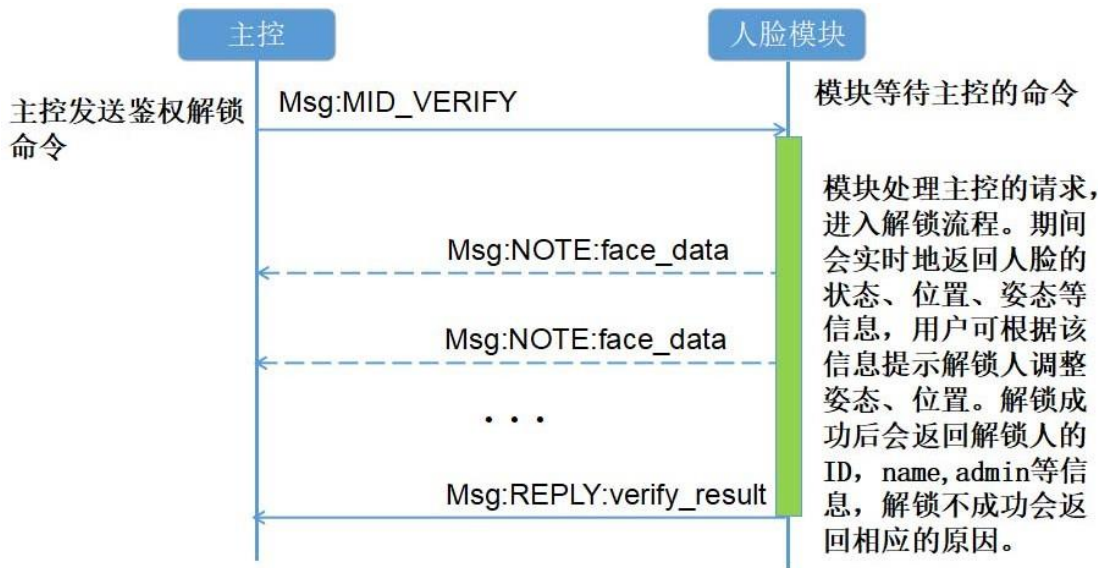


图 8

12 体验模式

如果客户需要在未注册有效用户的情况下体验人脸开锁效果，需要使用体验模式，可以采用如下两种方式：

1. 客户触发人脸识别后，锁板主控按照正常解锁流程下发识别指令给人脸模组，如收到 `MR_FAILED4_UNKNOWNUSER` 类型 `REPLY` 消息，表示人脸有效，体验模式下可解锁。其他类型 `REPLY` 和 `NOTE` 消息按需处理。此判断逻辑仅对真人有效，照片等假人无法解锁。但是仍然有鉴权处理，耗时稍长。
2. 客户触发人脸识别后，锁板主控先下发 `MID_DEMOMODE` 指令，人脸模组进入体验模式，然后再按照解锁流程下发解锁指令。体验模式下，只要有完整无遮挡人脸存在，无论照片，假体，真人都可触发解锁。

13 加密通信流程（可不使用，由客户决定）

加密通信流程如图 4-9 所示。

- ① 主控给模组上电。
- ② 模组发送 READY（明文）给主控。
- ③ 模组第一次开机需要调用 **MID_SET_RELEASE_ENC_KEY** 指令设置私有协议需要的 **16bytes** 序列（附件 2 中说明）。
- ④ 主控采用随机数生成随机序列并发送给模组（**4bytes**）。
- ⑤ 模组和主控双方根据这随机序列，采用自定义的私有协议（附件 2 中说明）生成 **16bytes** 的密码。双方采用相同算法生成相同的密码。之后双方本次会话都采用此密码加解密（**01：AES128 加密模式，02：使用异或取反加密模式**）。
- ⑥ 模组返回状态并采用随机密码 AES/SMPL 加密发送产品序列号给主控。
- ⑦ 主控解密，鉴别设备 ID 确认身份后开始处理需求指令，录入或解锁等。
- ⑧ 主控采用随机生成的密码，AES/SMPL 加密的方式对指令和数据加密并发送给模组。
- ⑨ 模组用第 4 步解密出来的密码解密，判断指令合法性并处理该指令。
- ⑩ 模组用第 4 步解密出来的密码对指令处理结果和数据加密，并发送给主控

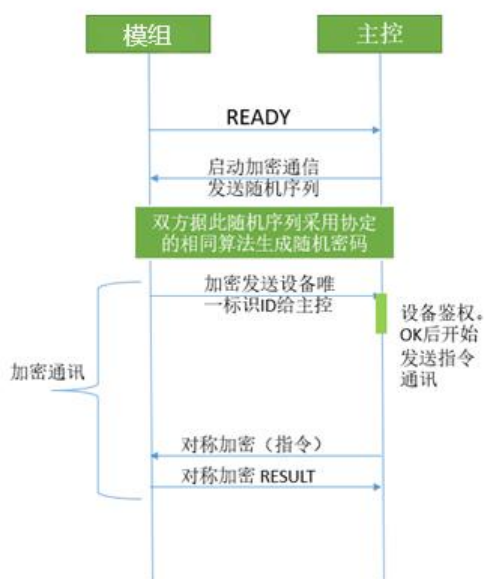


图 9

14 OTA 升级流程

- ① 主控发送 MID_START_OTA（加密通信）通知模组进入 OTA 模式；
- ② 模组反馈 OK（不加密通信，后面的都是不加密模式），表示进入了 OTA 模式；
- ③ 主控发送 MID_CONFIG_BAUDRATE 配置更高波特率，模组反馈 OK 后，主机同步设置相同波特率。延时 100ms 后才能开始串口数据传输。（该配置过程可选，波特率默认是 115200）；
- ④ 主控发送 MID_OTA_HEADER，把固件相关信息发送到模组；
- ⑤ 模组反馈 OK；
- ⑥ 主控发送 MID_GET_OTA_STATUS；
- ⑦ 模组反馈 OTA 状态和起始包序号；
- ⑧ 主控循环发送升级包 MID_OTA_PACKET，收到 OK 后，继续下一包；
- ⑨ 模组收完固件后，开始升级；
- ⑩ 模组升级结束反馈 NID_OTA_DONE。

15 补充说明

15.1 MID_REPLY

主控发送给模块的每条命令，最终都会收到模块的 MID_REPLY 回复，该消息包含主控发送的命令 mid，命令的执行结果 result，以及可能返回的数据 data。

```
typedef struct {  
    uint8_t mid; // the command(message) id to reply (the request message ID)  
    uint8_t result; // command handling result: success or failed -> s_msg_result  
    uint8_t data[0];  
} s_msg_reply_data;
```

命令的执行结果 result 可能是以下几种情况：

```
/* message result code */  
typedef uint8_t s_msg_result;  
const uint8_t MR_SUCCESS = 0; // success  
const uint8_t MR_REJECTED = 1; // module rejected this command  
const uint8_t MR_ABORTED = 2; // algo aborted  
const uint8_t MR_FAILED4_CAMERA = 4; // camera open failed  
const uint8_t MR_FAILED4_UNKNOWNREASON = 5; // UNKNOWN_ERROR  
const uint8_t MR_FAILED4_INVALIDPARAM = 6; // invalid param  
const uint8_t MR_FAILED4_NOMEMORY = 7; // no enough memory  
const uint8_t MR_FAILED4_UNKNOWNUSER = 8; // exceed limitation  
const uint8_t MR_FAILED4_MAXUSER = 9; // exceed maximum user number  
const uint8_t MR_FAILED4_FACEENROLLED = 10; // this face has been enrolled  
const uint8_t MR_FAILED4_LIVENESSCHECK = 12; // liveness check failed  
const uint8_t MR_FAILED4_TIMEOUT = 13; // exceed the time limit  
const uint8_t MR_FAILED4_AUTHORIZATION = 14; // authorization failed  
const uint8_t MR_FAILED4_READ_FILE = 19; // read file failed  
const uint8_t MR_FAILED4_WRITE_FILE = 20; // write file failed  
const uint8_t MR_FAILED4_NO_ENCRYPT = 21; // encrypt must be set  
/* message result code end */
```

返回的 data 根据 mid 而有所不同，在头文件 message.h 中，以“s_msg_reply_”开头的结构体都是各个指令发送 reply 消息时所携带的 data，以 verify 为例如下：

```
/* message reply data definitions */
typedef struct {
    uint8_t user_id_heb;
    uint8_t user_id_leb;
    uint8_t user_name[USER_NAME_SIZE];
    uint8_t admin;
    uint8_t unlockStatus;
} s_msg_reply_verify_data;
```

15.2 MID_NOTE

MSG::NOTE 为模块向主控主动上报的信息，例如当模块上电后会主动向主控发送一条 NOTE，表明自己已经 READY，可以接收主控的命令，主控收到该 NOTE 消息后即可开始发送录入或者解锁命令。NOTE 消息中所包含的数据为：

```
typedef struct
{
    uint8_t nid; // note id
    uint8_t data[0];
} s_msg_note_data;
/* module -> host note end */
```

15.3 MID_RESET

主控发送这条命令后，模块将取消之前正在执行的命令（例如录入、解锁等），返回 STANDBY 状态。

15.4 MID_GETSTATUS

主控发送该命令后，模块返回自身当前的状态，主要包括：

MS_STANDBY(0):模块处于空闲状态，等待主控命令

MS_BUSY(1):模块处于工作状态

MS_ERROR(2):模块出错，不能正常工作

MS_INVALID(3):模块未进行初始化

15.5 MID_VERIFY

MSG::VERIFY 为最常用功能，其携带的 data 如下：

```
// verify
typedef struct {
```

```

    uint8_t pd_rightaway; // power down right away after verifying
    uint8_t timeout; // timeout, unit second, default 10s
} s_msg_verify_data;

```

[pd_rightaway]表示是否在解锁后立刻关机； [timeout]为解锁超时时间（单位 s），默认为 10s，可以在发送解锁命令时设置，超时时间用户可以任意设定，最大不超过 255s。

解锁过程中，模块返回 NOTE 和 REPLY 两种消息。

NOTE 主要返回的是当前帧图片中人脸的状态，以及人脸位置和姿态。

REPLY 主要返回的是解锁过程执行完毕后的最终结果，可能返回的消息包括： MR_SUCCESS， MR_FAILED4_NOSUCHFACE， MR_ABORTED 等，详见表 7。当返回结果是 MR_SUCCESS 时，表示解锁成功。

数据结构如下：

```

typedef struct {
    uint8_t user_id_hcb;
    uint8_t user_id_lcb;
    uint8_t user_name[USER_NAME_SIZE];
    uint8_t admin;
    uint8_t unlockStatus;
} s_msg_reply_verify_data;

```

15.6 MID_ENROLL/MID_ENROLL_SINGLE

MSG::ENROLL 也是比较常用的功能，其附带的 data 如下：

```

// enroll user
typedef struct {
    uint8_t admin; // the user will be set to admin
    uint8_t user_name[32];
    s_face_dir face_direction;
    uint8_t timeout;
} s_msg_enroll_data;

```

[admin]指明该录入的人为管理员；

[timeout]为录入过程中的超时时间（单位 s），默认为 10s，可以在发送录入命令时设置，超时时间用户可以随意设置，最大不超过 255s；

[face_direction]仅在 MID_ENROLL 中有效，表示用户要录入的人脸方向，有效的人脸方向有 5 种，定义如下：

```

/* msg face direction */
typedef uint8_t s_face_dir;
const uint8_t FACE_DIRECTION_UP = 0x10; // face up
const uint8_t FACE_DIRECTION_DOWN = 0x08; // face down
const uint8_t FACE_DIRECTION_LEFT = 0x04; // face left
const uint8_t FACE_DIRECTION_RIGHT = 0x02; // face right

```

```
const uint8_t FACE_DIRECTION_MIDDLE = 0x01;    // face middle
const uint8_t FACE_DIRECTION_UNDEFINE = 0x00;  // face undefine
```

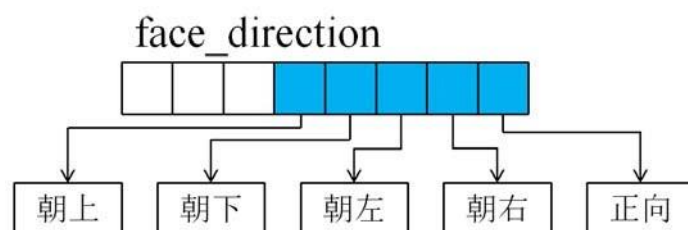
录入过程中同样也会返回 NOTE 和 REPLY 两种消息。

NOTE 主要返回的是当前帧图片中人脸的状态，以及人脸位置和姿态。

REPLY 主要返回录入过程执行完毕后的最终结果，可能返回的消息详见表 7。当返回 MR_SUCCESS 时，表示录入成功。REPLY 同时也会携带一些信息返回，如下所示：

```
typedef struct {
    uint8_t user_id_hcb;
    uint8_t user_id_lcb;
    uint8_t face_direction;
} s_msg_reply_enroll_data;
```

其中[face_direction]，数据的低 5 位从高到低分别表示人脸方向的朝上、朝下、朝左、朝右和正向，如下图所示，1 表示该朝向已录入，0 表示该朝向未录入。



15.7 MID_ENROLL_ITG

MSG::ENROLL_ITG 是对 1.15.6 ENROLL 命令的补充和扩展，录入类型可以在参数结构中指定，并可以让用户选择是否重复录入；其附带的 data 如下：

```
// enroll user intergrated
typedef struct {
    uint8_t admin; // the user will be set to admin, 1:admin user, 0:normal user
    uint8_t user_name[USER_NAME_SIZE];
    uint8_t face_direction; // reference FACE_DIRECTION_*
    uint8_t enroll_type; // reference FACE_ENROLL_TYPE_*
    uint8_t enable_duplicate; // enable user enroll duplicatly, 1:enable, 0:disable
    // when enroll_type is equal to FACE_ENROLL_TYPE_RGB, the
    enable_duplicate can be set to 2, it means that cant duplicate enroll with username
    uint8_t timeout; // timeout unit second default 10s
    uint8_t reserved[3]; // reserved feild
} s_msg_enroll_itg;
```

与 15.6 中描述一致的参数，定义不变

[enroll_type] FACE_ENROLL_TYPE_INTERACTIVE 时，为交互录入；

FACE_ENROLL_TYPE_SINGLE 为单帧录入

[enable_duplicate] 功能如下（user name / RGB 注册暂未支持）：

0：不可以同一个人录入，但是 user name 可以重复。

1：可以同一个人重复录入，user name 也可以重复。

2：可以同一个人录入，但是 user name 不可以重复(仅在 RGB 注册时有效)。

15.8 MID_ENROLL_BY_FEATURE

MSG::ENROLL_BY_FEATURE 其附带的 data 如下：

// enroll user

```
typedef struct {
    uint8_t admin; // the user will be set to admin
    uint8_t user_name[32]; // 暂未支持
    s_face_dir face_direction;
    uint8_t timeout;
    uint8_t feature[512];
} s_msg_enroll_feature_data;
```

[admin]指明该录入的人为管理员；

[timeout]为录入过程中的超时时间（单位 s），默认为 10s，可以在发送录入命令时设置，超时时间用户可以随意设置，最大不超过 255s；

[face_direction]不使用

[feature] 512 个字节的特征点

15.9 MID_DELUSER

MSG::DELUSER 为删除一个已注册的用户，通过 user_id 指定。

15.10 MID_DELALL

MSG::DELALL 为删除所有注册的用户信息。

15.11 MID_GETUSERINFO

主控通过 user id 指定要返回的注册用户，模块接收到该命令后会返回指定用户的信息，主要包括以下信息：

```
typedef struct {
    uint8_t user_id_hcb;
    uint8_t user_id_lcb;
    uint8_t user_name[32];
    uint8_t admin;
} s_msg_reply_getuserinfo_data;
```

15.12 MID_FACERESET

MSG::FACERESET 消息为重置 SDK 算法状态，如果正在进行交互式录入，发送此消息会重置录入的状态，已录入的方向全部被清空。

15.13 MID_POWERDOWN

主控给模块断电前需先发送 MID_POWERDOWN 命令，模块收到该命令后进行相应 log 保存机制操作，为了售后方便获取问题 log，**该消息在断电之前必须调用。**

15.14 MID_DEMOMODE

主控发送该命令后，模块进入演示模式，这种模式下模块鉴权的流程中将不做特征比对，即所有人都可以解锁，但是仍然会进行活体检测。

15.15 MID_GET_ALL_USERID

获取所有已注册用户的数量和所有已注册用户的 ID。

15.16 MID_SET_RELEASE_ENC_KEY

主控**在第一次开机时必须**发送该命令，模块会对该指令携带的数据加密并保存，之后模块会用该数据进行 4.14.18 中所描述的加密通信。该数据掉电不丢失。

15.17 MID_INIT_ENCRYPTION

主控下发随机数，模块在收到随机数后进行密码生成，并按照新密码发送一条 reply 的消息。

该结构体中除了随机数外，还包含加密 mode，以及设置系统时间的 crtime 数组，该数组存储的内容代表当前时区的秒数(1970 年至今)。

主控下发随机数的结构体，如下：

```
typedef struct {
    uint8_t seed[4];
    uint8_t mode;
    uint8_t crtime[4];
} s_msg_init_encryption_data;
```

其中加密 mode 现支持两种（默认使用 AES），结构体如下：

```
enum EncMode {
    ST_ENCMODE_DEFAULT = 0,
    ST_ENCMODE_AES = 1,
    ST_ENCMODE_SMPL = 2
};
```

模块加密完成后返回的数据结构体如下：

```
typedef struct {
```



```
uint8_t device_id[20];  
} s_msg_reply_init_encryption_data;
```

15.18 MID_SET_THRESHOLD_LEVEL

该接口用于修改算法安全等级， 在不同厂商的与用户的理解中， 对于安全的要求标准不一致， 所以开放安全等级接口， 用于设置 liveness 和 verify 的安全等级， 其中 0-4 分别代表： 低、较低、正常、较高、高。

系统默认安全等级为： 2 正常。主控下发结构体如下：

```
typedef struct {  
    uint8_t verify_threshold_level; // level 0~4, safety from low to high,  
    default 2  
    uint8_t liveness_threshold_level; // level 0~4, safety from low to high,  
    default 2  
} s_msg_algo_threshold_level;
```

声明： 推荐使用默认设置， 安全等级过高易导致通过率降低； 安全等级过低易导致误识问题， 请慎用该设置。

附件 1

```
#ifndef __FACE_LOCK_MESSAGE_H__
#define __FACE_LOCK_MESSAGE_H__
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdarg.h>

#define USER_NAME_SIZE 32
#define VERSION_INFO_BUFFER_SIZE 32
#define MAX_USER_COUNTS 20

/* communication message ID definitions*/
typedef uint8_t s_msg_id;

// Module to Host (m->h)
const uint8_t MID_REPLY = 0x00; // request(command) reply message, success with data or
fail with reason
const uint8_t MID_NOTE = 0x01; // note to host e.g. the position or angle of the face

// Host to Module (h->m)
const uint8_t MID_RESET = 0x10; // stop and clear all in-processing messages. enter standby
mode
const uint8_t MID_GETSTATUS = 0x11; // to ping the module and get the status
const uint8_t MID_VERIFY = 0x12; // to verify the person in front of the camera
const uint8_t MID_ENROLL = 0x13; // to enroll and register the person in front of the
camera
const uint8_t MID_ENROLL_SINGLE = 0x1D; // to enroll and register the person in front
of the camera, with one frame image
const uint8_t MID_DELUSER = 0x20; // Delete the specified user with user id
const uint8_t MID_DELLALL = 0x21; // Delete all registered users
const uint8_t MID_GETUSERINFO = 0x22; // Get user info
const uint8_t MID_FACERESET = 0x23; // Reset face status
const uint8_t MID_GET_ALL_USERID = 0x24; // get all users ID
const uint8_t MID_ENROLL_ITG = 0x26; // Integrated enroll message,support all
existing enroll type
const uint8_t MID_GET_VERSION = 0x30; // get version information
const uint8_t MID_START_OTA = 0x40; // ask the module to enter OTA mode
const uint8_t MID_STOP_OTA = 0x41; // ask the module to exit OTA mode
const uint8_t MID_GET_OTA_STATUS = 0x42; // query the current ota status
const uint8_t MID_OTA_HEADER = 0x43; // the ota header data
const uint8_t MID_OTA_PACKET = 0x44; // the data packet, carries real firmware data
```

```

const uint8_t MID_INIT_ENCRYPTION = 0x50; // initialize encrypted communication
const uint8_t MID_CONFIG_BAUDRATE = 0x51; // config uart baudrate
const uint8_t MID_SET_RELEASE_ENC_KEY = 0x52; // set release encrypted key
(Warning!!!:Once set, the KEY will not be able to modify)
const uint8_t MID_SET_DEBUG_ENC_KEY = 0x53; // set debug encrypted key
const uint8_t MID_SET_THRESHOLD_LEVEL = 0xD4; // Set threshold level
const uint8_t MID_POWERDOWN = 0xED; // be prepared to power off
const uint8_t MID_DEMOMODE = 0xFE; // enter demo mode, verify flow will skip feature
comparation step.
const uint8_t MID_MAX = 0xFF; // reserved
/* communication message ID definitions end */

```

```

/* message result code */

```

```

typedef uint8_t s_msg_result;
const uint8_t MR_SUCCESS = 0; // success
const uint8_t MR_REJECTED = 1; // module rejected this command
const uint8_t MR_ABORTED = 2; // algo aborted
const uint8_t MR_FAILED4_CAMERA = 4; // camera open failed
const uint8_t MR_FAILED4_UNKNOWNREASON = 5; //UNKNOWN_ERROR
const uint8_t MR_FAILED4_INVALIDPARAM = 6; // invalid param
const uint8_t MR_FAILED4_NOMEMORY = 7; // no enough memory
const uint8_t MR_FAILED4_UNKNOWNUSER = 8; // no user enrolled
const uint8_t MR_FAILED4_MAXUSER = 9; // exceed maximum user number
const uint8_t MR_FAILED4_FACEENROLLED = 10; // this face has been enrolled
const uint8_t MR_FAILED4_LIVENESSCHECK = 12; // liveness check failed
const uint8_t MR_FAILED4_TIMEOUT = 13; // exceed the time limit
const uint8_t MR_FAILED4_AUTHORIZATION = 14; // authorization failed
const uint8_t MR_FAILED4_CAMERAFOV = 15; // camera fov test failed
const uint8_t MR_FAILED4_CAMERAQUA = 16; // camera quality test failed
const uint8_t MR_FAILED4_CAMERASTRU = 17; // camera structure test failed
const uint8_t MR_FAILED4_BOOT_TIMEOUT = 18; // boot up timeout
const uint8_t MR_FAILED4_READ_FILE = 19; // read file failed
const uint8_t MR_FAILED4_WRITE_FILE = 20; // write file failed
const uint8_t MR_FAILED4_NO_ENCRYPT = 21; // encrypt must be set
/* message result code end */

```

```

/* module state */

```

```

typedef uint8_t s_mstate;
const uint8_t MS_STANDBY = 0; // IDLE, waiting for commands
const uint8_t MS_BUSY = 1; // in working of processing commands
const uint8_t MS_ERROR = 2; // in error state. can't work properly
const uint8_t MS_INVALID = 3; // not initialized
const uint8_t MS_OTA = 4; // in ota state
/* module state end */

```

```

// a general message adapter
typedef struct {
    uint8_t mid; // the message id
    uint8_t size_heb; // high eight bits
    uint8_t size_leb; // low eight bits
    uint8_t data[0];
} s_msg;

/* message data definitions */
/* module -> host note */
/* msg note id */
typedef uint8_t s_note_id;
const uint8_t NID_READY = 0; // module ready for handling request (command)
const uint8_t NID_FACE_STATE = 1; // the detected face status description
const uint8_t NID_UNKNOWNERROR = 2; // unknown error
const uint8_t NID_OTA_DONE = 3; // OTA upgrading processing done
/* msg note id end */

/* msg face direction */
typedef uint8_t s_face_dir;
const uint8_t FACE_DIRECTION_UP = 0x10; // face up
const uint8_t FACE_DIRECTION_DOWN = 0x08; // face down
const uint8_t FACE_DIRECTION_LEFT = 0x04; // face left
const uint8_t FACE_DIRECTION_RIGHT = 0x02; // face right
const uint8_t FACE_DIRECTION_MIDDLE = 0x01; // face middle
const uint8_t FACE_DIRECTION_UNDEFINE = 0x00; // face undefine
/* msg face direction end */

const uint8_t FACE_STATE_NORMAL = 0; // normal state, the face is available
const uint8_t FACE_STATE_NOFACE = 1; // no face detected
const uint8_t FACE_STATE_TOOUP = 2; // face is too up side
const uint8_t FACE_STATE_TOODOWN = 3; // face is too down side
const uint8_t FACE_STATE_TOOLEFT = 4; // face is too left side
const uint8_t FACE_STATE_TOORIGHT = 5; // face is too right side
const uint8_t FACE_STATE_TOOFAR = 6; // face is too far
const uint8_t FACE_STATE_TOO CLOSE = 7; // face is too near
const uint8_t FACE_STATE_EYEBROW_OCCLUSION = 8; // eyebrow occlusion
const uint8_t FACE_STATE_EYE_OCCLUSION = 9; // eye occlusion
const uint8_t FACE_STATE_FACE_OCCLUSION = 10; // face occlusion
const uint8_t FACE_STATE_DIRECTION_ERROR = 11; // face direction error

typedef struct {

```

```

int16_t state; // corresponding to FACE_STATE_*

// position
int16_t left; // in pixel
int16_t top;
int16_t right;
int16_t bottom;
// pose
int16_t yaw; // up and down in vertical orientation
int16_t pitch; // right or left turned in horizontal orientation
int16_t roll; // slope
} s_note_data_face;

typedef struct {
    uint8_t nid; // note id
    uint8_t data[0];
} s_msg_note_data;
/* module -> host note end */

// enroll user
typedef struct {
    uint8_t admin; // the user will be set to admin
    uint8_t user_name[USER_NAME_SIZE];
    s_face_dir face_direction;
    uint8_t timeout; // timeout, unit second default 10s
} s_msg_enroll_data;

// enroll user intergrated
typedef struct {
    uint8_t admin; // the user will be set to admin, 1:admin user,0:normal user
    uint8_t user_name[USER_NAME_SIZE];
    uint8_t face_direction; // reference FACE_DIRECTION_*
    uint8_t enroll_type; // reference FACE_ENROLL_TYPE_*
    uint8_t enable_duplicate; // enable user enroll duplicatly, 1:enable,0:disable
    uint8_t timeout; // timeout unit second default 10s
    uint8_t reserved[3]; // reserved feild
} s_msg_enroll_itg;

// verify
typedef struct {
    uint8_t pd_rightaway; // power down right away after verifying
    uint8_t timeout; // timeout, unit second, default 10s
} s_msg_verify_data;

```

```

// delete user
typedef struct {
    uint8_t user_id_heb; // high eight bits of user_id to be deleted
    uint8_t user_id_leb; // low eight bits pf user_id to be deleted
} s_msg_deluser_data;

// get user info
typedef struct {
    uint8_t user_id_heb; // high eight bits of user_id to get info
    uint8_t user_id_leb; // low eight bits of user_id to get info
} s_msg_getuserinfo_data;

// start ota data MID_START_OTA
typedef struct {
    uint8_t v_primary; // primary version number
    uint8_t v_secondary; // secondary version number
    uint8_t v_revision; // revision number
} s_msg_startota_data;

// ota header MID_OTA_HEADER
typedef struct {
    uint8_t fsize_b[4]; // OTA FW file size int -> [b1, b2, b3, b4]
    uint8_t num_pkt[4]; // number packet to be divided for transferring, int ->[b1, b2, b3, b4]
    uint8_t pkt_size[2]; // raw data size of single packet
    uint8_t md5_sum[32]; // md5 check sum
} s_msg_otaheader_data;

// ota packet MID_OTA_PACKET
typedef struct {
    uint8_t pid[2]; // the packet id
    uint8_t psize[2]; // the size of this package
    uint8_t data[0]; // data 0 start
} s_msg_otapacket_data;

// MID_INIT_ENCRYPTION data
typedef struct {
    uint8_t seed[4]; // random data as a seed
    uint8_t mode; // reserved for selecting encrytion mode
    uint8_t crttime[4];
} s_msg_init_encryption_data;

// demo data
typedef struct {
    uint8_t enable; // enable demo or not

```

```
} s_msg_demomode_data;
```

```
/* message reply data definitions */
```

```
typedef struct {  
    uint8_t user_id_heb;  
    uint8_t user_id_leb;  
    uint8_t user_name[USER_NAME_SIZE];  
    uint8_t admin;  
    uint8_t unlockStatus;  
} s_msg_reply_verify_data;
```

```
typedef struct {  
    uint8_t user_id_heb;  
    uint8_t user_id_leb;  
    uint8_t face_direction;  
} s_msg_reply_enroll_data;
```

```
typedef struct {  
    uint8_t status;  
} s_msg_reply_getstatus_data;
```

```
typedef struct {  
    uint8_t version_info[VERSION_INFO_BUFFER_SIZE];  
} s_msg_reply_version_data;
```

```
typedef struct {  
    uint8_t user_id_heb;  
    uint8_t user_id_leb;  
    uint8_t user_name[USER_NAME_SIZE];  
    uint8_t admin;  
} s_msg_reply_getuserinfo_data;
```

```
typedef struct {  
    uint8_t user_counts; // number of enrolled users  
    uint8_t users_id[MAX_USER_COUNTS*2]; //use 2 bytes to save a user ID and save  
    high eight bits firstly  
} s_msg_reply_all_userid_data;
```

```
typedef struct {  
    uint8_t time_heb; // high eight bits of factory test time  
    uint8_t time_leb; // low eight bits of facatory test time  
} s_msg_reply_factory_test_data;
```

```

// MID_INIT_ENCRYPTION reply
typedef struct {
    uint8_t device_id[20]; // the unique ID of this device, string type
} s_msg_reply_init_encryption_data;

// REPLY MID_GET_OTA_STATUS
typedef struct {
    uint8_t ota_status; // current ota status
    uint8_t next_pid_e[2]; // expected next pid, [b0,b1]
} s_msg_reply_getotastatus_data;

typedef struct {
    uint8_t mid; // the command(message) id to reply (the request message ID)
    uint8_t result; // command handling result: success or failed ->s_msg_result
    uint8_t data[0];
} s_msg_reply_data;

typedef struct {
    uint8_t ota_result; // 0 :is sucess; 1: is faile;
} s_note_ota_result;

typedef struct {
    uint8_t baudrate_index; // 1: is 115200 (115200*1); 2 is 230400 (115200*2); 3 is 460800
    (115200*4); 4 is 1500000
} s_msg_config_baudrate;

typedef struct {
    uint8_t enc_key_number[16];
} s_msg_enc_key_number_data;

#endif // __FACE_LOCK_MESSAGE_H__

```


附件 2

该文档在调试串口通讯加密时使用

1. HOST 发送密钥抽取规则

用户自行拟定 16Bytes 数据作为密钥的抽取规则。

如拟定抽取规则为：0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

调试阶段使用：SET_DEBUG_ENC_KEY(0x53)

Host Send: EFAA 53 0010 000102030405060708090A0B0C0D0E0F 43

Module Reply: EFAA 00 0002 53 00 51 // 00 表示 SUCCESS

或正式出厂使用：SET_RELEASE_ENC_KEY(0x52)

Host Send: EFAA 52 0010 000102030405060708090A0B0C0D0E0F 42

Module Reply: EFAA 00 0002 52 00 50 // 00 表示 SUCCESS

注：

SET_DEBUG_ENC_KEY(0x53)和 SET_RELEASE_ENC_KEY(0x52)区别如下：

用户调试阶段可使用 SET_DEBUG_ENC_KEY(0x53)设置加密通信密钥抽取规则，该设置只对本次会话有效，重启后消失。

出厂时调用模组 SET_RELEASE_ENC_KEY(0x52)指令一次性设置进入模组保存，出厂前调用一次即可。

2. HOST 发送随机序列生成密钥

- 用户可采用时间作为种子生成一个 4Bytes 任意随机序列，可定义为 char data[4]

如：3C 56 B3 85(Hex)

Host Send: EFAA 50 0009 3C 56 B3 85 ** ##### \$\$

注：

** 表示加密模式，01：AES128 加密模式，02：使用异或取反加密模式

表示 4Bytes 时间戳

密钥的生成仅和 3C 56 B3 85 这 4Bytes 数据有关，和** #####无关。

- 4bytes 的随机序列转换为字符生成 md5 (32 个字符序列)，生成方法见文末参考代码及 md5sum.cpp

如：3C 56 B3 85(Hex)生成 32 个字符序列的 md5 为：

ee71 5353 57ad 9bb4 7cb6 e1ba 638c 26a6

[可采用 echo -ne '\x3c\x56\xb3\x85' |md5sum|cut -d ' ' -f1 得到 md5sum，结合验证]

- 按上文中用户指定的密钥抽取规则从生成的 32 个字符序列中抽取其中 16 个作为通讯密钥。

对应密钥抽取规则（抽取 MD5 的对应位）：0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

如按照上面设置的秘钥抽取规则抽取的通讯密钥为：ee71535357ad9bb4

后续通讯使用 ee71535357ad9bb4 采用指定加密模式对下发指令和回复消息加密解密。

3. 参考代码

➤ 设置密钥抽取规则

```
s_status SenseControl::setKey(uint8_t *seed, int size) {  
    // the input seed must be 4 bytes  
    if (size != 4) {  
        LOGE("Incorrect input seed size");  
        return INVALID_PARAM;  
    }  
    MD5 * md5 = new MD5(seed, size);  
    //生成 md5  
    const char *md5str = md5->toString().c_str();  
    LOGD("md5sum: str = %s", md5str);  
    // special code, 此处的密钥抽取规则为: [7 13 20 4 13 1 23 7 5 9 0 27 11 24 4 10]  
    snprintf(mEncKey, ENC_KEY_SIZE+1, "%c%c%c%c%c%c%c%c%c%c%c%c%c%c%c%c",  
        md5str[7], md5str[13], md5str[20], md5str[4],  
        md5str[13], md5str[1], md5str[23], md5str[7],  
        md5str[5], md5str[9], md5str[0], md5str[27],  
        md5str[11], md5str[24], md5str[4], md5str[10]);  
    LOGD("mEncKey:%s", mEncKey);  
    return OK;  
}
```

➤ **md5sum.h**

```
#ifndef MD5_H
#define MD5_H

#include <string>
#include <fstream>

/* Type define */
typedef unsigned char byte;
typedef unsigned int uint32;

using std::string;
using std::ifstream;

/* MD5 declaration. */
class MD5 {
public:
    MD5();
    MD5(const void* input, size_t length);
    MD5(const string& str);
    MD5(ifstream& in);
    void update(const void* input, size_t length);
    void update(const string& str);
    void update(ifstream& in);
    const byte* digest();
    string toString();
    void reset();

private:
    void update(const byte* input, size_t length);
    void final();
    void transform(const byte block[64]);
    void encode(const uint32* input, byte* output, size_t length);
    void decode(const byte* input, uint32* output, size_t length);
    string bytesToHexString(const byte* input, size_t length);

    /* class uncopyable */
    MD5(const MD5&);
    MD5& operator=(const MD5&);

private:
    uint32 _state[4];    /* state (ABCD) */
```

```

uint32 _count[2];    /* number of bits, modulo 2^64 (low-order word first) */
byte _buffer[64];    /* input buffer */
byte _digest[16];    /* message digest */
bool _finished;      /* calculate finished ? */

static const byte PADDING[64];    /* padding for calculate */
static const char HEX[16];
enum { BUFFER_SIZE = 1024 };
};

#endif /*MD5_H*/

```

➤ **md5sum.cpp**

```
#include "md5sum.h"
#include <string.h>

using namespace std;

/* Constants for MD5Transform routine. */
#define S11 7
#define S12 12
#define S13 17
#define S14 22
#define S21 5
#define S22 9
#define S23 14
#define S24 20
#define S31 4
#define S32 11
#define S33 16
#define S34 23
#define S41 6
#define S42 10
#define S43 15
#define S44 21

/* F, G, H and I are basic MD5 functions.
*/
#define F(x, y, z) (((x) & (y)) | ((~x) & (z)))
#define G(x, y, z) (((x) & (z)) | ((y) & (~z)))
#define H(x, y, z) ((x) ^ (y) ^ (z))
#define I(x, y, z) ((y) ^ ((x) | (~z)))

/* ROTATE_LEFT rotates x left n bits.
*/
#define ROTATE_LEFT(x, n) (((x) << (n)) | ((x) >> (32-(n))))

/* FF, GG, HH, and II transformations for rounds 1, 2, 3, and 4.
Rotation is separate from addition to prevent recomputation.
*/
#define FF(a, b, c, d, x, s, ac) { \
    (a) += F ((b), (c), (d)) + (x) + ac; \
    (a) = ROTATE_LEFT ((a), (s)); \
    (a) += (b); \
}
```

```

}
#define GG(a, b, c, d, x, s, ac) { \
    (a) += G ((b), (c), (d)) + (x) + ac; \
    (a) = ROTATE_LEFT ((a), (s)); \
    (a) += (b); \
}
#define HH(a, b, c, d, x, s, ac) { \
    (a) += H ((b), (c), (d)) + (x) + ac; \
    (a) = ROTATE_LEFT ((a), (s)); \
    (a) += (b); \
}
#define II(a, b, c, d, x, s, ac) { \
    (a) += I ((b), (c), (d)) + (x) + ac; \
    (a) = ROTATE_LEFT ((a), (s)); \
    (a) += (b); \
}

const byte MD5::PADDING[64] = { 0x80 };
const char MD5::HEX[16] = {
    '0', '1', '2', '3',
    '4', '5', '6', '7',
    '8', '9', 'a', 'b',
    'c', 'd', 'e', 'f'
};

/* Default construct. */
MD5::MD5() {
    reset();
}

/* Construct a MD5 object with a input buffer. */
MD5::MD5(const void* input, size_t length) {
    reset();
    update(input, length);
}

/* Construct a MD5 object with a string. */
MD5::MD5(const string& str) {
    reset();
    update(str);
}

```

```

/* Construct a MD5 object with a file. */
MD5::MD5(istream& in) {
    reset();
    update(in);
}

/* Return the message-digest */
const byte* MD5::digest() {

    if (!_finished) {
        _finished = true;
        final();
    }
    return _digest;
}

/* Reset the calculate state */
void MD5::reset() {

    _finished = false;
    /* reset number of bits. */
    _count[0] = _count[1] = 0;
    /* Load magic initialization constants. */
    _state[0] = 0x67452301;
    _state[1] = 0xefcdab89;
    _state[2] = 0x98badcfe;
    _state[3] = 0x10325476;
}

/* Updating the context with a input buffer. */
void MD5::update(const void* input, size_t length) {
    update((const byte*)input, length);
}

/* Updating the context with a string. */
void MD5::update(const string& str) {
    update((const byte*)str.c_str(), str.length());
}

/* Updating the context with a file. */
void MD5::update(istream& in) {

    if (!in) {
        return;
    }
}

```

```

    }

    std::streamsize length;
    char buffer[BUFFER_SIZE];
    while (!in.eof()) {
        in.read(buffer, BUFFER_SIZE);
        length = in.gcount();
        if (length > 0) {
            update(buffer, length);
        }
    }
    in.close();
}

/* MD5 block update operation. Continues an MD5 message-digest operation, processing another
message block, and updating the context. */

void MD5::update(const byte* input, size_t length) {

    uint32 i, index, partLen;

    _finished = false;

    /* Compute number of bytes mod 64 */
    index = (uint32)((_count[0] >> 3) & 0x3f);

    /* update number of bits */
    if ((_count[0] += ((uint32)length << 3)) < ((uint32)length << 3)) {
        ++_count[1];
    }
    _count[1] += ((uint32)length >> 29);

    partLen = 64 - index;

    /* transform as many times as possible. */
    if (length >= partLen) {

        memcpy(&_buffer[index], input, partLen);
        transform(_buffer);

        for (i = partLen; i + 63 < length; i += 64) {
            transform(&input[i]);
        }
        index = 0;
    }
}

```



```

    } else {
        i = 0;
    }

    /* Buffer remaining input */
    memcpy(&_buffer[index], &input[i], length - i);
}

/* MD5 finalization. Ends an MD5 message-_digest operation, writing the the message _digest and
zeroizing the context. */
void MD5::final() {

    byte bits[8];
    uint32 oldState[4];
    uint32 oldCount[2];
    uint32 index, padLen;

    /* Save current state and count. */
    memcpy(oldState, _state, 16);
    memcpy(oldCount, _count, 8);

    /* Save number of bits */
    encode(_count, bits, 8);

    /* Pad out to 56 mod 64. */
    index = (uint32)((_count[0] >> 3) & 0x3f);
    padLen = (index < 56) ? (56 - index) : (120 - index);
    update(PADDING, padLen);

    /* Append length (before padding) */
    update(bits, 8);

    /* Store state in digest */
    encode(_state, _digest, 16);

    /* Restore current state and count. */
    memcpy(_state, oldState, 16);
    memcpy(_count, oldCount, 8);
}

/* MD5 basic transformation. Transforms _state based on block. */
void MD5::transform(const byte block[64]) {

```

```
uint32 a = _state[0], b = _state[1], c = _state[2], d = _state[3], x[16];
```

```
decode(block, x, 64);
```

```
/* Round 1 */
```

```
FF (a, b, c, d, x[ 0], S11, 0xd76aa478); /* 1 */
FF (d, a, b, c, x[ 1], S12, 0xe8c7b756); /* 2 */
FF (c, d, a, b, x[ 2], S13, 0x242070db); /* 3 */
FF (b, c, d, a, x[ 3], S14, 0xc1bdceee); /* 4 */
FF (a, b, c, d, x[ 4], S11, 0xf57c0faf); /* 5 */
FF (d, a, b, c, x[ 5], S12, 0x4787c62a); /* 6 */
FF (c, d, a, b, x[ 6], S13, 0xa8304613); /* 7 */
FF (b, c, d, a, x[ 7], S14, 0xfd469501); /* 8 */
FF (a, b, c, d, x[ 8], S11, 0x698098d8); /* 9 */
FF (d, a, b, c, x[ 9], S12, 0x8b44f7af); /* 10 */
FF (c, d, a, b, x[10], S13, 0xffff5bb1); /* 11 */
FF (b, c, d, a, x[11], S14, 0x895cd7be); /* 12 */
FF (a, b, c, d, x[12], S11, 0x6b901122); /* 13 */
FF (d, a, b, c, x[13], S12, 0xfd987193); /* 14 */
FF (c, d, a, b, x[14], S13, 0xa679438e); /* 15 */
FF (b, c, d, a, x[15], S14, 0x49b40821); /* 16 */
```

```
/* Round 2 */
```

```
GG (a, b, c, d, x[ 1], S21, 0xf61e2562); /* 17 */
GG (d, a, b, c, x[ 6], S22, 0xc040b340); /* 18 */
GG (c, d, a, b, x[11], S23, 0x265e5a51); /* 19 */
GG (b, c, d, a, x[ 0], S24, 0xe9b6c7aa); /* 20 */
GG (a, b, c, d, x[ 5], S21, 0xd62f105d); /* 21 */
GG (d, a, b, c, x[10], S22, 0x2441453); /* 22 */
GG (c, d, a, b, x[15], S23, 0xd8a1e681); /* 23 */
GG (b, c, d, a, x[ 4], S24, 0xe7d3fbc8); /* 24 */
GG (a, b, c, d, x[ 9], S21, 0x21e1cde6); /* 25 */
GG (d, a, b, c, x[14], S22, 0xc33707d6); /* 26 */
GG (c, d, a, b, x[ 3], S23, 0xf4d50d87); /* 27 */
GG (b, c, d, a, x[ 8], S24, 0x455a14ed); /* 28 */
GG (a, b, c, d, x[13], S21, 0xa9e3e905); /* 29 */
GG (d, a, b, c, x[ 2], S22, 0xfcefa3f8); /* 30 */
GG (c, d, a, b, x[ 7], S23, 0x676f02d9); /* 31 */
GG (b, c, d, a, x[12], S24, 0x8d2a4c8a); /* 32 */
```

```
/* Round 3 */
```

```
HH (a, b, c, d, x[ 5], S31, 0xfffa3942); /* 33 */
HH (d, a, b, c, x[ 8], S32, 0x8771f681); /* 34 */
HH (c, d, a, b, x[11], S33, 0x6d9d6122); /* 35 */
```

```

HH (b, c, d, a, x[14], S34, 0xfde5380c); /* 36 */
HH (a, b, c, d, x[ 1], S31, 0xa4beea44); /* 37 */
HH (d, a, b, c, x[ 4], S32, 0x4bdecfa9); /* 38 */
HH (c, d, a, b, x[ 7], S33, 0xf6bb4b60); /* 39 */
HH (b, c, d, a, x[10], S34, 0xbebfb7c70); /* 40 */
HH (a, b, c, d, x[13], S31, 0x289b7ec6); /* 41 */
HH (d, a, b, c, x[ 0], S32, 0xea127fa); /* 42 */
HH (c, d, a, b, x[ 3], S33, 0xd4ef3085); /* 43 */
HH (b, c, d, a, x[ 6], S34, 0x4881d05); /* 44 */
HH (a, b, c, d, x[ 9], S31, 0xd9d4d039); /* 45 */
HH (d, a, b, c, x[12], S32, 0xe6db99e5); /* 46 */
HH (c, d, a, b, x[15], S33, 0x1fa27cf8); /* 47 */
HH (b, c, d, a, x[ 2], S34, 0xc4ac5665); /* 48 */

/* Round 4 */
II (a, b, c, d, x[ 0], S41, 0xf4292244); /* 49 */
II (d, a, b, c, x[ 7], S42, 0x432aff97); /* 50 */
II (c, d, a, b, x[14], S43, 0xab9423a7); /* 51 */
II (b, c, d, a, x[ 5], S44, 0xfc93a039); /* 52 */
II (a, b, c, d, x[12], S41, 0x655b59c3); /* 53 */
II (d, a, b, c, x[ 3], S42, 0x8f0ccc92); /* 54 */
II (c, d, a, b, x[10], S43, 0xffeff47d); /* 55 */
II (b, c, d, a, x[ 1], S44, 0x85845dd1); /* 56 */
II (a, b, c, d, x[ 8], S41, 0x6fa87e4f); /* 57 */
II (d, a, b, c, x[15], S42, 0xfe2ce6e0); /* 58 */
II (c, d, a, b, x[ 6], S43, 0xa3014314); /* 59 */
II (b, c, d, a, x[13], S44, 0x4e0811a1); /* 60 */
II (a, b, c, d, x[ 4], S41, 0xf7537e82); /* 61 */
II (d, a, b, c, x[11], S42, 0xbd3af235); /* 62 */
II (c, d, a, b, x[ 2], S43, 0x2ad7d2bb); /* 63 */
II (b, c, d, a, x[ 9], S44, 0xeb86d391); /* 64 */

_state[0] += a;
_state[1] += b;
_state[2] += c;
_state[3] += d;
}

/* Encodes input (ulong) into output (byte). Assumes length is
a multiple of 4.
*/
void MD5::encode(const uint32* input, byte* output, size_t length) {

    for (size_t i = 0, j = 0; j < length; ++i, j += 4) {

```

```

        output[j]= (byte)(input[i] & 0xff);
        output[j + 1] = (byte)((input[i] >> 8) & 0xff);
        output[j + 2] = (byte)((input[i] >> 16) & 0xff);
        output[j + 3] = (byte)((input[i] >> 24) & 0xff);
    }
}

/* Decodes input (byte) into output (ulong). Assumes length is
a multiple of 4.
*/
void MD5::decode(const byte* input, uint32* output, size_t length) {

    for (size_t i = 0, j = 0; j < length; ++i, j += 4) {
        output[i] = (((uint32)input[j]) | (((uint32)input[j + 1]) << 8) |
            (((uint32)input[j + 2]) << 16) | (((uint32)input[j + 3]) << 24));
    }
}

/* Convert byte array to hex string. */
string MD5::bytesToHexString(const byte* input, size_t length) {

    string str;
    str.reserve(length << 1);
    for (size_t i = 0; i < length; ++i) {
        int t = input[i];
        int a = t / 16;
        int b = t % 16;
        str.append(1, HEX[a]);
        str.append(1, HEX[b]);
    }
    return str;
}

/* Convert digest to string value */
string MD5::toString() {
    return bytesToHexString(digest(), 16);
}

```